ALPHA ARTS AND SCIENCE COLLEGE

TEAM NM ID :- 69a69f5fc3309e5746ec1ab9

PROJECT NAME:-MICRO GOAL

TEAM LEAD:-KARTHIKEYAN S

NM ID- unm1429212403206

TEAM MEMBER:- ASHWITHA V

NM ID- UNM1429212403192

TEAM MEMBER:-:- VIJAYALAKSHMI S

NM ID- UNM1429212403228

TEAM MEMBER:-:- PRASANNA KUMARI D

NM ID- UNM1429212403216

Introduction :

The MicroGoal Goal Tracker Backend API is developed using Node.js, Express.js, and MongoDB.

This project allows users to manage their personal goals securely. Users can register, login, create goals, update them, view them, and delete them.

The backend system uses JWT (JSON Web Token) authentication to ensure that only authorized users can access and manage their goals.

This project demonstrates the implementation of RESTful APIs, database integration, authentication, and CRUD operations.

Objectives :

The main objectives of this project are:

- To build a secure backend API using Node.js and Express.js
- To store and manage data using MongoDB
- To implement JWT authentication for security
- To perform CRUD operations on user goals
- To test APIs using Thunder Client

Technologies Used :

Node.js – Used as the runtime environment for executing JavaScript on the server side.

Express.js – Used as the backend framework for building APIs.

MongoDB – A NoSQL database used to store user and goal data.

JWT (JSON Web Token) – Used for authentication and secure access to protected routes.

Thunder Client – Used to test API requests inside Visual Studio Code.

Project Setup :

The backend server is initialized in the index.js file.

Main steps involved in project setup include:

• Creating an Express server

• Connecting the server to MongoDB database

• Defining API routes

• Implementing authentication middleware

• Handling errors properly

Environment variables are stored in a .env file which includes:

• MONGO_URI – MongoDB connection string

• JWT_SECRET – secret key for token generation

• PORT – server port number

Using environment variables improves the security and flexibility of the application.

Database Models

The project uses two main database models.

User Model:

The User schema contains the following fields:
• Name
• Email
• Password

The password is securely stored after encryption.

Goal Model:
The Goal schema contains:
• Title of the goal
• Completed status (true/false)
• User reference (to identify the owner of the goal)

This ensures that each user can only manage their own goals.

API Routes :

The application provides several API endpoints.

User Routes:
POST /api/users – Used for user registration and login.

Goal Routes (Protected):

GET /api/goals – Retrieve all goals of a user.

POST /api/goals – Create a new goal.

PUT /api/goals/:id – Update an existing goal.

DELETE /api/goals/:id – Delete a goal.

These routes are protected using JWT authentication.

Goal Creation (Screenshot Reference):

In the below screenshot, the user successfully creates a goal using a POST request.

POST http://localhost:8000/create Send

Query Headers 2 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "title": "Learn NodeJS"
}
```

Status: 200 OK Size: 141 Bytes Time: 103 ms

Response Headers 7 Cookies Results Docs { }

```
{
  "success": true,
  "data": {
    "title": "Learn NodeJS",
    "completed": false,
    "user": "69aa6f98ebb7dcca33be4d98",
    "_id": "69aa6ffcebb7dcca33be4d9b",
    "__v": 0
  }
}
```

Response Chart

The request includes:

• Goal title

• Completed status

After login and token generation, the user sends a request to the backend API to create a goal.

POST http://localhost:8000/login Send

Query Headers 2 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{

  "email":"gskarthikkrishnan01@gmail.com",
  "password":"123456"
}
```

Status: 200 OK Size: 423 Bytes Time: 283 ms

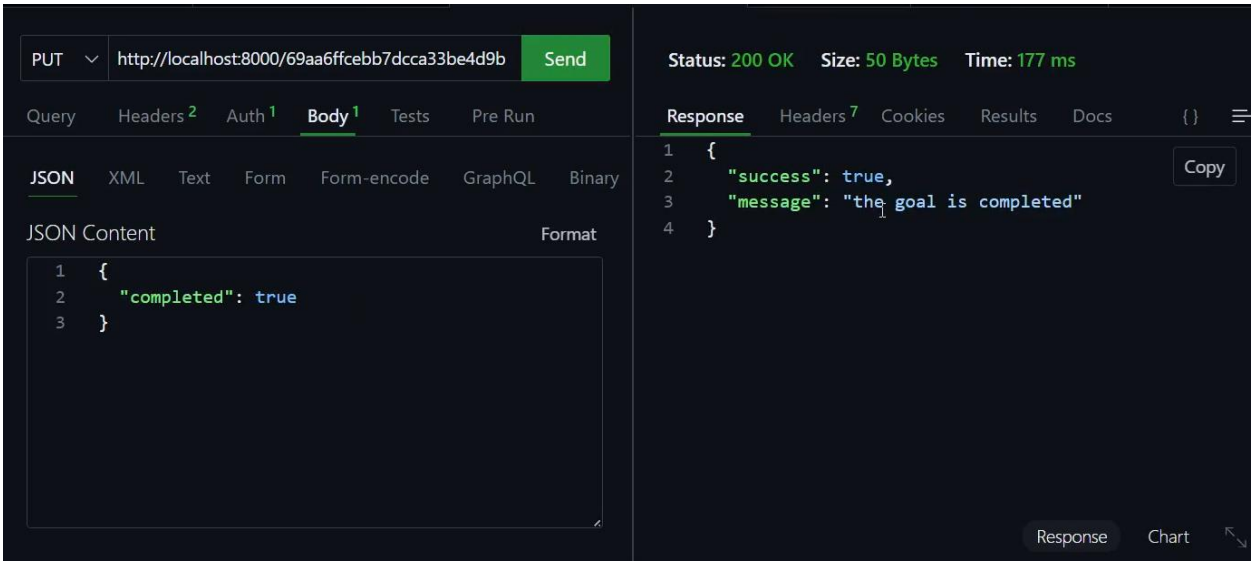
Response Headers 7 Cookies Results Docs { }

Copy

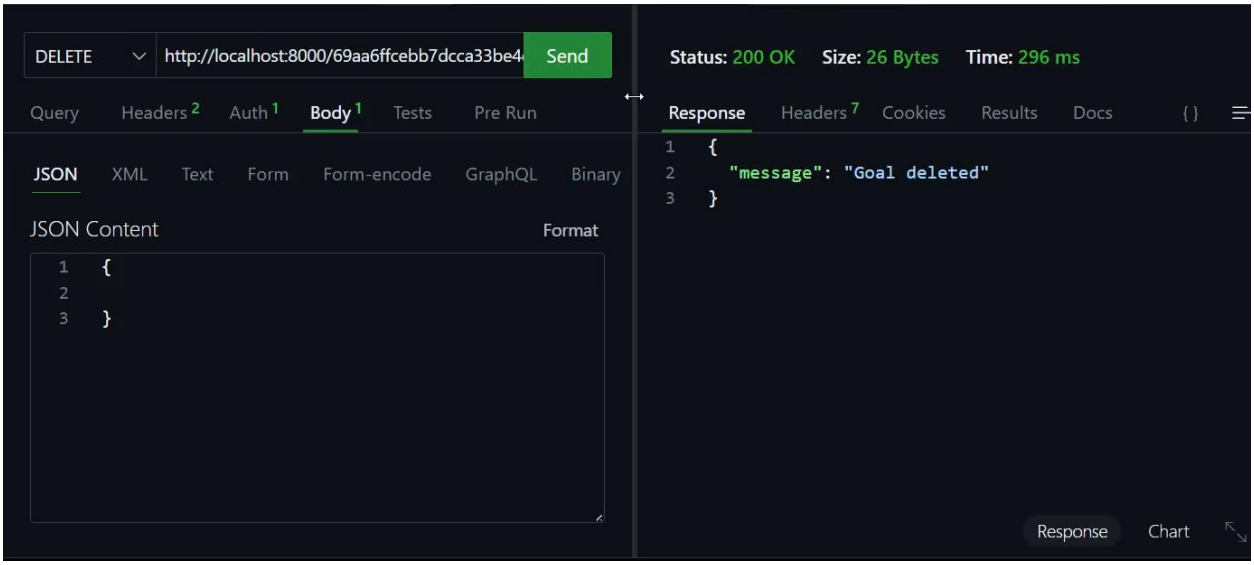

```
{
  "success": true,
  "message": "login succesfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJ1c2VySWQiOiI2OWFhNmY5OGViYjdkY2NhMzNiZTR
      kOTgiLCJpYXQiOjE3NzI3NzczODIsImV4cCI6MTc3Mjg
      2Mzc4Mn0.JxDSY9c5_GtyAAhlvPvKgAz
      -_TrbtI8sSgdTqyD3-o0",
  "data": {
    "_id": "69aa6f98ebb7dcca33be4d98",
    "name": "karthik krishnan",
    "email": "gskarthikkrishnan01@gmail.com",
    "password": "$2b$10$dRkxcFjxskpC/4ivURpoM
        .noIJJXKvAkMjXXUvQMuQ289Ls16TEDC",
```

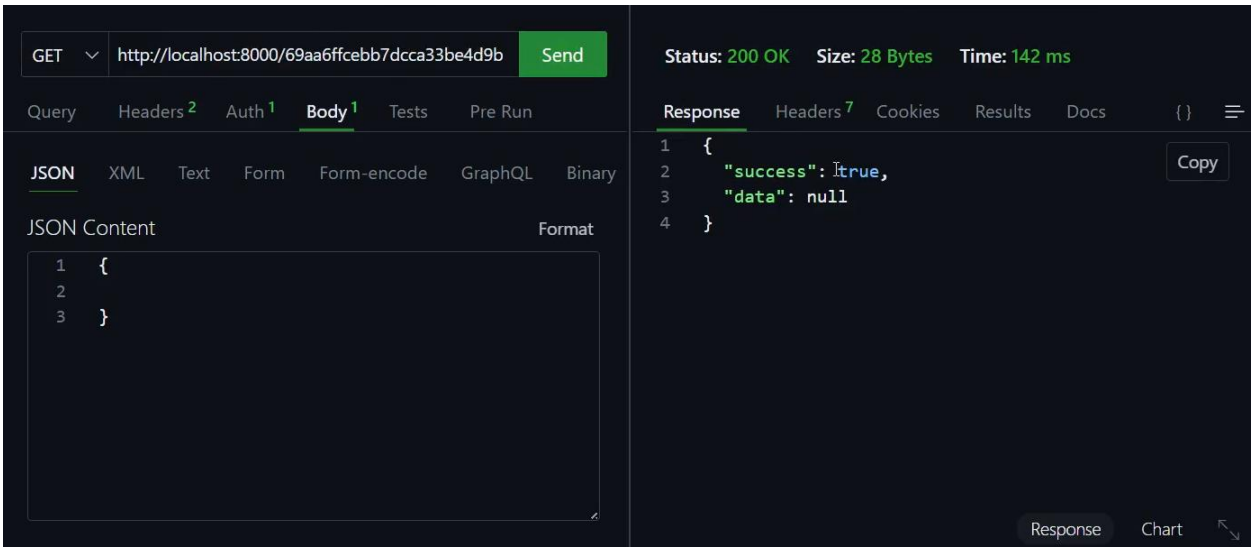

Response Chart

Update Goal:


PUT
http://localhost:8000/69aa6ffcebb7dcca33be4d9b
Send
Status: 200 OK
Size: 50 Bytes
Time: 177 ms
Query
Headers 2
Auth 1
Body 1
Tests
Pre Run
Response
Headers 7
Cookies
Results
Docs
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
Copy
{
"completed": true
}
{
"success": true,
"message": "the goal is completed"
}
Response
Chart

Delete Goal:


DELETE
http://localhost:8000/69aa6ffcebb7dcca33be4
Send
Status: 200 OK
Size: 26 Bytes
Time: 296 ms
Query
Headers 2
Auth 1
Body 1
Tests
Pre Run
Response
Headers 7
Cookies
Results
Docs
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
}
{
"message": "Goal deleted"
}
Response
Chart


GET
http://localhost:8000/69aa6ffcebb7dcca33be4d9b
Send
Query
Headers 2
Auth 1
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
}
Status: 200 OK
Size: 28 Bytes
Time: 142 ms
Response
Headers 7
Cookies
Results
Docs
{ }
Copy
{
"success": true,
"data": null
}
Response
Chart